GPU and multithreaded performance of SIESTA-REMAG for spin–orbit calculations on hcp cobalt

SIESTA-REMAG [1] (REMAG below) is a modified SIESTA 5.4.2 [2, 3] that runs the self-consistent field (SCF) cycle on one NVIDIA GPU. Its main target is noncollinear magnetism with spin–orbit coupling. We timed REMAG against unmodified SIESTA on the same inputs and machine, for a two-atom cobalt cell sampled with up to 4,400 k points. With 4,400 k points, REMAG on one H100 reaches SCF convergence in a median 14.5 s, against 66.4 s for unmodified SIESTA on 28 CPUs with MPI. Both produce identical results to every printed digit.

Test calculation

Every timing uses bulk hcp cobalt with two atoms per cell ($a = 2.507$ Å, $c = 4.069$ Å). Spin–orbit coupling uses SIESTA's off-site form [4], which keeps the coupling between orbitals on different atoms. The pseudopotential is the fully relativistic PseudoDojo nc-fr-04 PBE standard set [5], read in PSML format [6], with 3s and 3p semicore states in the valence. The basis is triple-ζ with polarization (energy shift 0.005 Ry). That gives 50 orbitals per cell, so each k point has a 100×100 complex Hamiltonian once both spin components are included. The real-space mesh cutoff is 1,200 Ry. Exchange and correlation use PBE [7] with SIESTA's standard noncollinear treatment of the gradient terms. Occupations follow Fermi–Dirac smearing at 2,000 K. Both atomic moments start at 1.6 μ_B, tilted 45° from the c axis, and are free to relax. The high cutoff and temperature come from the magnetic-anisotropy study this input was built for. They make the mesh work and the smearing larger than in a typical production run.

An SCF run counts as converged when no density-matrix element changes by more than 10^{-7} and no Hamiltonian element by more than 10^{-4} eV between iterations. Mixing is Pulay's scheme [8] with weight 0.05 and eight stored steps. A tighter Hamiltonian tolerance of 10^{-5} eV cannot be met on every mesh. On the $6 \times 6 \times 3$ mesh, unmodified SIESTA stalls with the largest change fixed at 1.4×10^{-5} eV while the energy is constant to 1 μeV. REMAG's GPU runs did not converge within 400 iterations there or on the $14 \times 14 \times 8$ mesh. Convergence here is SCF convergence at a fixed k mesh, not convergence in the number of k points.

The Brillouin zone is sampled with Monkhorst–Pack meshes [9] of $6 \times 6 \times 3$, $10 \times 10 \times 6$, $14 \times 14 \times 8$ and $20 \times 20 \times 11$ points. With spin–orbit coupling SIESTA does not use time-reversal symmetry to reduce them, so these give 108, 600, 1,568 and 4,400 k points. Every k point is diagonalized in every SCF iteration.

What was timed

We timed five configurations on the same input files.

1. Unmodified SIESTA 5.4.2 on one CPU. This is the numerical reference.
2. Unmodified SIESTA 5.4.2 with MPI on 28 processes. With `Diag.ParallelOverK`, each process diagonalizes its own share of the k points. The real-space mesh work is also shared out.
3. REMAG on 28 CPU threads. The k-point diagonalizations are shared among the threads. The mesh work stays on one thread.
4. REMAG on the GPU with its block-Jacobi eigensolver. After a Cholesky reduction, this solver repeatedly diagonalizes pairs of 32×32 blocks until the off-diagonal part is negligible. The SCF cycle, the initial density on the mesh and the final forces all run on the GPU. Groups of 64 k points are solved together, each starting from the previous iteration's eigenvectors.

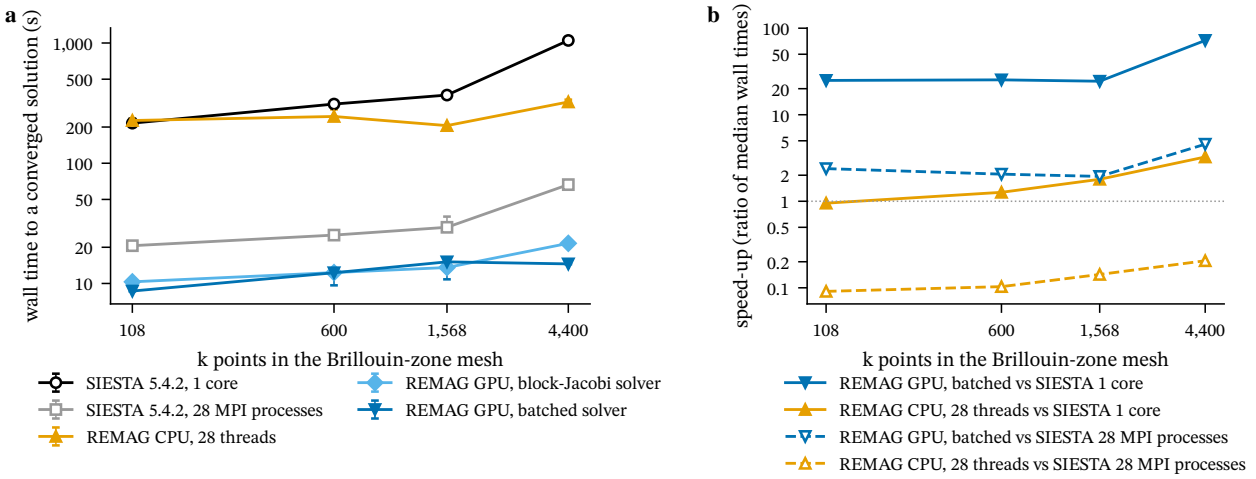


Figure 1. Time to SCF convergence against the number of k points. (a) Wall time of each configuration: medians of three runs, with bars from the fastest to the slowest. SIESTA on one CPU was run once on the two largest meshes. (b) Unmodified SIESTA's median time divided by REMAG's, against SIESTA on one CPU (filled symbols, solid lines) and on 28 MPI processes (open symbols, dashed lines). Above the dotted line at 1, REMAG is faster.

5. REMAG on the GPU with a batched dense eigensolver. This is the same as the previous configuration, except for the solver. Each group of 256 k points is reduced with batched Cholesky and triangular solves and then diagonalized with cuSOLVER's `cusolverDnXsyevBatched` [10].

Everything runs in double precision. Unmodified SIESTA diagonalizes each k point twice per iteration with LAPACK's divide-and-conquer solver. The first solve gives eigenvalues to place the Fermi level, and the second gives eigenvectors. In the timer records this is 35,200 solves for 4,400 k points over four iterations. REMAG on the CPU keeps the eigenvectors from one solve in memory. The REMAG settings used here are all optional, and REMAG's defaults were not timed.

Each timing is the wall-clock time of one fresh process, from launch to exit. It covers reading the input, setup of the basis and mesh, the SCF cycle, the final forces and stress, and the output files. Runs never overlapped, and repetitions of the five configurations were interleaved. The machine had one NVIDIA H100 80GB HBM3 (driver 595.91.07, CUDA 12.9). The container was given 28 CPUs' worth of time on a host with two Intel Xeon Platinum 8480+ sockets, 112 cores and 224 hardware threads, shared with other tenants. Processes and threads were not pinned to cores, and GPU runs used one CPU thread. GPU clocks were not locked. All CPU code was built with GCC 13.3 against OpenBLAS 0.3.26 using one BLAS thread. The MPI build used Open MPI 4.1.6 and ScaLAPACK 2.2.1. SIESTA's OpenMP support was off in every build.

Results

With 4,400 k points, REMAG on the GPU with the batched solver takes 14.5 s (Figure 1a). That is 4.6 times faster than unmodified SIESTA on 28 MPI processes and 72 times faster than SIESTA on one CPU, a figure that rests on a single one-CPU run. On the three smaller meshes the GPU is 1.9 to 2.4 times faster than MPI SIESTA (Figure 1b). The ratio jumps at 4,400 k points because MPI SIESTA's time more than doubles, from 29.3 s to 66.4 s, while the GPU time stays between 8.6 s and 15.1 s on every mesh. The number of SCF iterations differs between meshes, 17 on the $14 \times 14 \times 8$ mesh against 22 to 24 on the others (Table 1). That is why every curve dips there. On any one mesh, every configuration took the same number of iterations. The block-Jacobi solver is within two seconds of the batched one up to 1,568 k points, and takes 21.6 s with 4,400.

REMAG on 28 CPU threads is slower than MPI SIESTA on every mesh: 323 s against 66.4 s with 4,400 k points. MPI SIESTA divides the real-space mesh work among its processes, and REMAG leaves it on one thread. REMAG on the CPU is 3.2 times faster than SIESTA on one CPU with 4,400 k points, and no faster with 108.

With the batched solver the SCF cycle takes 6.0 s of the run, less than the 7.3 s of setup (Figure 2a). With the block-Jacobi solver the SCF cycle is still the largest part, 13.6 s. For unmodified SIESTA it is nearly the whole run. The final forces take about 0.3 s on the GPU against about 20 s for SIESTA and REMAG on one CPU thread. Setup is shorter for the GPU runs than for REMAG on the CPU because the first Hamiltonian is built on the GPU. Within the SCF cycle, the batched solver cuts the time to build and diagonalize the Hamiltonians from 9.5 s to

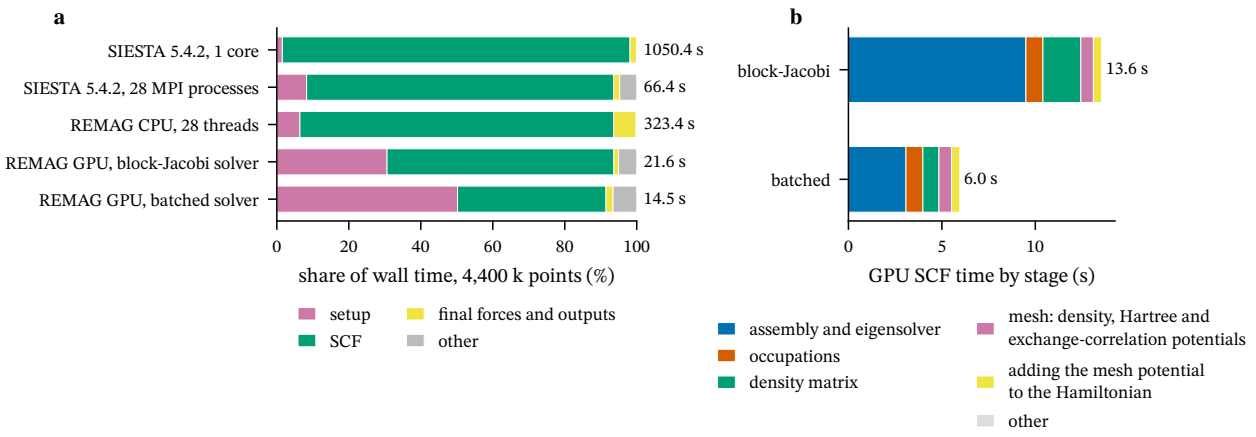


Figure 2. Where the time goes with 4,400 k points. (a) The median run of each configuration divided into setup (basis, neighbour lists, real-space mesh and the first Hamiltonian), the SCF cycle, and the final forces and output files. "Other" is process start-up and input handling. The total is printed beside each bar. (b) The GPU SCF cycle divided into its stages. These come from one additional run of each GPU solver in which the stages ran one after another and were clocked separately.

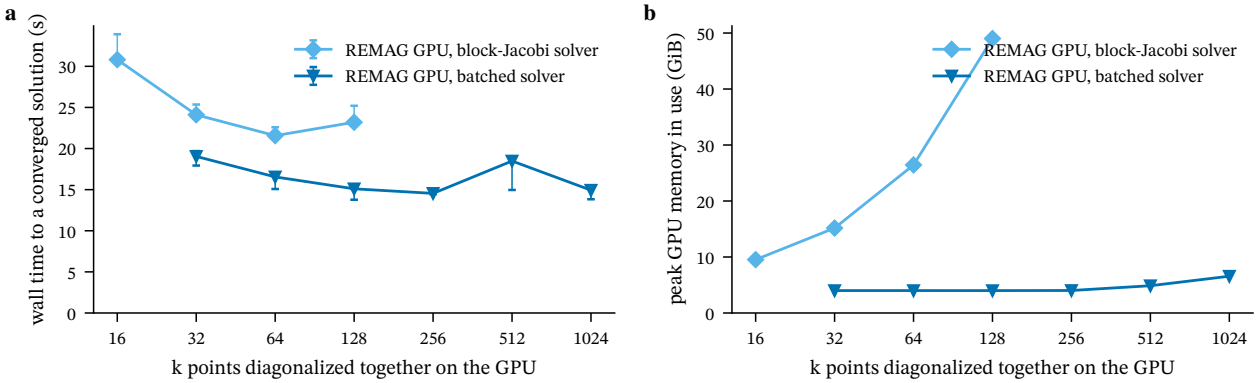


Figure 3. Effect of the number of k points diagonalized together, with 4,400 k points in total. (a) Wall time to SCF convergence, median and range of three runs. (b) Peak GPU memory in use, read from the driver every 0.25 s. The block-Jacobi solver accepts at most 128 k points at once. Groups of 16 were not run with the batched solver.

3.1 s (Figure 2b). Building the density matrix falls from 2.0 s to 0.9 s, because a whole group of k points is added into it in one pass instead of one pass per k point.

How many k points are diagonalized together trades speed against memory (Figure 3). The block-Jacobi solver is fastest with 64 at once. Its memory rises steadily with the group size, from 9.5 GiB at 16 to 49.0 GiB at 128. The batched solver takes between 14.5 s and 15.1 s with 128, 256 or 1,024 at once, and never uses more than 6.6 GiB. With 512 at once, two of three runs took about 18.5 s. We did not investigate this.

REMAG's CPU threads handle the diagonalizations faster than SIESTA at low CPU counts (Figure 4). On one CPU, REMAG takes 14.27 s per iteration against SIESTA's 34.99 s. Most of that gap is SIESTA's second diagonalization of every k point. REMAG gains little beyond eight threads: 4.01 s on 8 and 3.06 s on 28. MPI SIESTA goes from 5.32 s on 8 processes to 2.58 s on 28. That is about twice the ideal value at 28, and it overtakes REMAG between 8 and 16. We have not measured which part of REMAG's threaded code limits it.

Every configuration produced the same converged state (Table 1). This holds to the last printed digit: 1 µeV for energies and $10^{-5}\,\mu_B$ for the moment. The stress tensor also agreed to every printed digit. The forces are zero by symmetry. The 26 extra runs behind Figures 2b and 3 gave the same values.

REMAG also runs three treatments that unmodified SIESTA lacks (Figure 5). In these runs, REMAG's spin constraint, built on the Lagrange-multiplier scheme of Cuadrado *et al.* [11], holds the moment direction with a field on the atomic moments. The local-frame form of PBE evaluates the gradient terms of the functional in the local spin direction at each mesh point, the approach Kübler *et al.* introduced for the local density approximation [12]. It takes 15.1 s on the GPU against 337 s on 28 CPU threads. The multicollinear form of Pu *et al.* [13] averages the collinear functional over 64 spin directions. It takes 15.1 s against 859 s. Optimized tetrahedron occupations [14] replace smearing by an integration over tetrahedra of k points, with local-frame PBE. They take 15.8 s against 381 s. For each treatment the GPU and CPU runs print the same final energies, Fermi level and

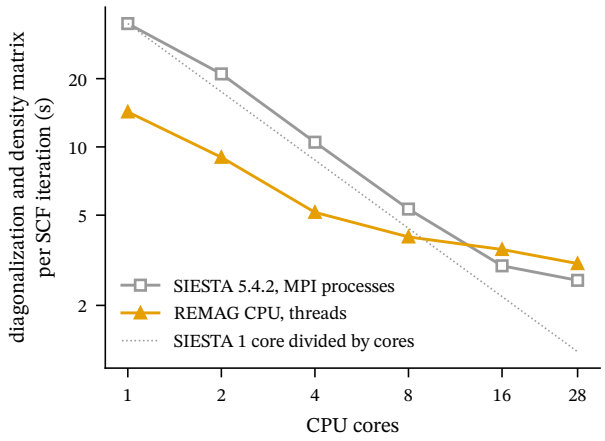


Figure 4. Wall time per SCF iteration spent diagonalizing the 4,400 k points and building the density matrix, against the number of CPUs. The runs stopped after four iterations. Points are single runs for SIESTA and the mean of two runs for REMAG. The dotted line is SIESTA on one CPU divided by the number of CPUs. Early iterations cost more than the average over a full run. For MPI SIESTA on 28 processes, these runs give 2.58 s per iteration against 1.87 s to 2.13 s in the converged runs.

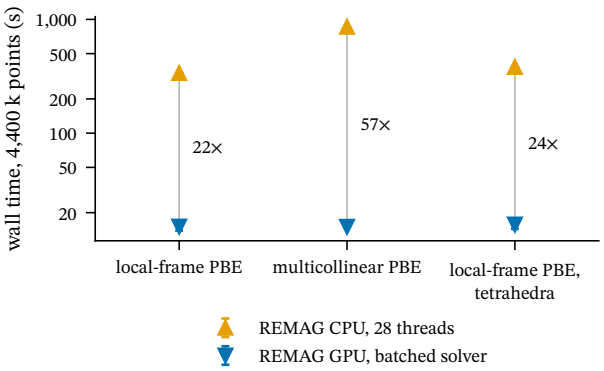


Figure 5. Wall time for three treatments that unmodified SIESTA does not offer, with 4,400 k points and the moment direction held at 45° from the c axis. Points are REMAG on 28 CPU threads (one run) and on the GPU with the batched solver (median and range of three runs). The printed numbers are the ratios.

Table 1. Converged results on each mesh. Every run of every configuration on that mesh printed exactly these values and took this number of SCF iterations. Energies and the Fermi level are in eV, and the magnitude of the cell's spin moment is in μ_B.

Mesh	k points	Runs	Iterations	Total energy	Free energy	Fermi level	Moment
$6 \times 6 \times 3$	108	15	22	−8,326.464264	−8,326.640830	−5.097158	3.22154
$10 \times 10 \times 6$	600	15	23	−8,326.396679	−8,326.603890	−5.152804	3.27037
$14 \times 14 \times 8$	1,568	13	17	−8,326.383615	−8,326.598894	−5.149439	3.23967
$20 \times 20 \times 11$	4,400	13	24	−8,326.388186	−8,326.600084	−5.146938	3.22885

moment. With tetrahedra the GPU took 25 iterations and the CPU 28. These ratios compare with REMAG on 28 CPU threads, which is itself about five times slower than MPI SIESTA on the plain input. They do not measure the GPU against well-parallelized CPU code.

Scope

All of these numbers come from one small metallic cell with many k points, on one machine. Each k point carries a small matrix, so the work is many independent small eigenproblems plus a fixed amount of mesh work. Larger cells with fewer k points load the code differently and were not timed. Nor was a GPU build of unmodified SIESTA that offloads the eigensolver through ELPA [15]. The CPU comparison uses 28 CPUs' worth of time out of a 112-core host, without pinning, and is not normalized for cost or energy.

Data and scripts

Everything needed to check these numbers is in the SIESTA-REMAG source tree, in the folder `benchmarks/gpu-performance-hcp-co`. `data/records.jsonl` gives each run's wall time, iteration count and final results. `scripts/bench.py` generated the inputs and ran every calculation, and `scripts/analyze.py` drew every figure and computed every number quoted here. `data/machine.txt` has the SHA-256 hashes of the three SIESTA executables and of REMAG's GPU library. The REMAG build is commit `2b8a2c0` of SIESTA-REMAG plus uncommitted changes, identified by those hashes. Full inputs, outputs and SIESTA timer trees for every run are kept in a separate archive of about 1 GB.

References

[1] SIESTA-REMAG contributors. SIESTA-REMAG: GPU acceleration of SIESTA for magnetic materials. `https://github.com/willgbryan/SIESTA-REMAG`, 2026. Commit 2b8a2c0 with local changes.

[2] José M. Soler, Emilio Artacho, Julian D. Gale, Alberto García, Javier Junquera, Pablo Ordejón, and Daniel Sánchez-Portal. The SIESTA method for ab initio order-N materials simulation. *Journal of Physics: Condensed Matter*, 14(11):2745–2779, 2002. doi: 10.1088/0953-8984/14/11/302.

[3] Alberto García, Nick Papior, Arsalan Akhtar, Emilio Artacho, et al. SIESTA: Recent developments and applications. *The Journal of Chemical Physics*, 152:204108, 2020. doi: 10.1063/5.0005077.

[4] R. Cuadrado and J. I. Cerdá. Fully relativistic pseudopotential formalism under an atomic orbital basis: spin–orbit splittings and magnetic anisotropies. *Journal of Physics: Condensed Matter*, 24(8):086005, 2012. doi: 10.1088/0953-8984/24/8/086005.

[5] M. J. van Setten, M. Giantomassi, E. Bousquet, M. J. Verstraete, D. R. Hamann, X. Gonze, and G.-M. Rignanese. The PseudoDojo: Training and grading a 85 element optimized norm-conserving pseudopotential table. *Computer Physics Communications*, 226:39–54, 2018. doi: 10.1016/j.cpc.2018.01.012.

[6] Alberto García, Matthieu J. Verstraete, Yann Pouillon, and Javier Junquera. The psml format and library for norm-conserving pseudopotential data curation and interoperability. *Computer Physics Communications*, 227:51–71, 2018. doi: 10.1016/j.cpc.2018.02.011.

[7] John P. Perdew, Kieron Burke, and Matthias Ernzerhof. Generalized gradient approximation made simple. *Physical Review Letters*, 77:3865–3868, 1996. doi: 10.1103/PhysRevLett.77.3865.

[8] Péter Pulay. Convergence acceleration of iterative sequences. The case of SCF iteration. *Chemical Physics Letters*, 73:393–398, 1980. doi: 10.1016/0009-2614(80)80396-4.

[9] Hendrik J. Monkhorst and James D. Pack. Special points for Brillouin-zone integrations. *Physical Review B*, 13:5188–5192, 1976. doi: 10.1103/PhysRevB.13.5188.

[10] NVIDIA Corporation. cuSOLVER library. `https://docs.nvidia.com/cuda/cusolver/`, 2025. CUDA Toolkit 12.9.

[11] Ramón Cuadrado, Miguel Pruneda, Alberto García, and Pablo Ordejón. Implementation of non-collinear spin-constrained DFT calculations in SIESTA with a fully relativistic Hamiltonian. *Journal of Physics: Materials*, 1(1):015010, 2018. doi: 10.1088/2515-7639/aae7db.

[12] J. Kübler, K.-H. Höck, J. Sticht, and A. R. Williams. Density functional theory of non-collinear magnetism. *Journal of Physics F: Metal Physics*, 18(3):469–483, 1988. doi: 10.1088/0305-4608/18/3/018.

[13] Zhichen Pu, Hao Li, Ning Zhang, Hong Jiang, Yiqin Gao, Yunlong Xiao, Qiming Sun, Yong Zhang, and Sihong Shao. Noncollinear density functional theory. *Physical Review Research*, 5:013036, 2023. doi: 10.1103/PhysRevResearch.5.013036.

[14] Mitsuaki Kawamura, Yoshihiro Gohda, and Shinji Tsuneyuki. Improved tetrahedron method for the Brillouin-zone integration applicable to response functions. *Physical Review B*, 89:094515, 2014. doi: 10.1103/PhysRevB.89.094515.

[15] A. Marek, V. Blum, R. Johanni, V. Havu, B. Lang, T. Auckenthaler, A. Heinecke, H.-J. Bungartz, and H. Lederer. The ELPA library: scalable parallel eigenvalue solutions for electronic structure theory and computational science. *Journal of Physics: Condensed Matter*, 26(21):213201, 2014. doi: 10.1088/0953-8984/26/21/213201.